# SIMATS ENGINEERING

## JAVA PROGRAMMING — ASSESSMENT 1

**Student: M. Naga Shalini**
**Register Number: 192524183**

### TASK 1 — Understand the Problem and Identify the Objects

The system receives a vehicle registration number, vehicle category, entry time, and (for an electric vehicle) charging information. It must find a suitable free slot, allocate the slot, issue a unique ticket, record the entry time, calculate the parking fee when the vehicle exits, release the slot, and report when the facility is full. The main output includes the allocated floor and slot, ticket details, fee, and success/failure messages.

**Candidate classes and responsibilities**

ParkingFacility owns the floors and slots, searches for a free slot, allocates it, releases it, and tracks active parking. ParkingSlot stores its floor/slot number and whether it is occupied. Vehicle stores common details such as registration number and entry time. Car, TwoWheeler and ElectricCar represent vehicle-specific behaviour and fee rules. ParkingTicket stores a unique ticket number, vehicle, slot, entry time and exit time. Chargeable is an interface for vehicles that use EV charging. A separate ParkingService may coordinate entry and exit operations.

**Business rules and assumptions**

A slot can contain at most one vehicle. A vehicle registration number must not have two active tickets at once. Ticket numbers must be unique and never reused during the program run. A vehicle can exit only if it has an active ticket. A slot index and floor index must be valid. If no compatible slot is available, allocation fails without changing the facility. Parking duration cannot be negative. The sample implementation assumes slots are searched from the first floor and first slot, and that all vehicle categories can use any free slot unless the requirements add category-specific slot restrictions.

**Alternative: parallel arrays**

A parallel-array design could store registration numbers in String[] regNo, vehicle types in String[] type, and entry times in long[] entryTime. Searching by registration number takes O(n) time for n vehicles. Adding another attribute requires adding and synchronising another array, which increases the chance of mismatched indexes. A Vehicle object groups related data and behaviour together; it also supports inheritance and polymorphism. Both approaches need O(n) storage, but the object-oriented design is easier to extend and maintain.

## TASK 2 — Choose the Data Representation

Use int for floor numbers, slot numbers, and counts; String for registration and ticket IDs; long for epoch-millisecond timestamps; and double or preferably long integer paise for money. Using integer paise avoids binary floating-point rounding in billing. A jagged array such as String[][] slots is appropriate because each floor can have a different number of slots. A null value means the slot is free; a non-null value stores the vehicle registration number.

Input can be read using java.util.Scanner for a simple console application. Convert minutes to hours carefully: if every started hour is charged, use (minutes + 59) / 60 with integer arithmetic. For example, 61 minutes gives (61 + 59) / 60 = 2 hours. The expression minutes / 60 would incorrectly charge 1 hour for 61 minutes. For fractional hourly billing, convert to double before division, e.g. minutes / 60.0. Validate all input before changing the system state.

### Alternative representation

A one-dimensional array can store slots using index = floor × slotsPerFloor + slot only when every floor has the same number of slots. If floor sizes differ, that formula can map to the wrong location. A jagged array directly represents the real structure and makes checking a floor's valid slot range straightforward. An ArrayList<String> can grow dynamically but requires a separate mapping to translate floor/slot into a list index. Both arrays use O(total slots) space; locating a slot by scanning takes O(total slots) time.

## TASK 3 — Design the Control Logic

### Allocation pseudocode

```
FOR each floor f
    FOR each slot s on floor f
        IF slot[f][s] is free
            store vehicle number in slot[f][s]
            RETURN {f, s}
RETURN null  // no free slot
```

### Release pseudocode

```
IF floor is outside valid floor range
    RETURN false
IF slot is outside valid range for that floor
    RETURN false
```

```
IF slot[floor][slot] is already free
   RETURN false
set slot[floor][slot] to null
RETURN true
```

Fee calculation should first validate that exit time is not before entry time. Calculate the duration in minutes, convert it to billable hours using the stated policy, then apply the vehicle category's rate and any daily maximum. Use if–else or switch for a small fixed set of fee slabs. A break statement exits a loop once a suitable slot is found; return exits the method immediately. If the facility is full, return null and do not overwrite any occupied slot.

**Completed ParkingFacility code**

```
import java.util.Arrays;

public class ParkingFacility {
   private final String[][] slots;

   public ParkingFacility(int[] slotsPerFloor) {
      slots = new String[slotsPerFloor.length][];
      for (int f = 0; f < slotsPerFloor.length; f++) {
         slots[f] = new String[slotsPerFloor[f]];
      }
   }

   public int[] allocate(String vehicleNo) {
      if (vehicleNo == null || vehicleNo.trim().isEmpty()) return null;
      for (int f = 0; f < slots.length; f++) {
         for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
               slots[f][s] = vehicleNo;
               return new int[]{f, s};
            }
         }
      }
      return null;
   }

   public boolean release(int floor, int slot) {
```

```
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1)
                + " " + p.release(5, 0));
        System.out.println(Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

**Expected output**

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

## TASK 4 — Design Classes, Objects and Constructors

Use constructors to create valid objects with their required fields. Vehicle can hold common data such as registration number and entry time; subclasses add their own details. ParkingTicket holds ticket number, vehicle, slot and entry timestamp. ParkingFacility creates its slot array from the supplied floor sizes. Use this(...) to chain constructors and avoid repeating validation or initialisation. Use super(...) to initialise the superclass part of a subclass.

A default constructor is useful only when a valid empty object can exist. A parameterised constructor is better when a vehicle must always have a registration number. A copy constructor is optional; it should not duplicate a unique ticket number

when ticket uniqueness is a business rule. Object references are local variables or fields that refer to objects; objects generally live on the heap and become eligible for garbage collection when no reachable reference remains.

## TASK 5 — Apply Static Members and Object Life Cycle

A static ticket counter belongs to the class because all tickets need unique numbers. A static method may generate the next ticket ID or report a class-wide count. An instance variable would create a separate counter for every facility, which could cause duplicate ticket numbers when two gates issue tickets independently. If ticket IDs need only be unique within each facility, use an instance counter plus a facility identifier; if they must be globally unique, use a shared static counter or a UUID. After a vehicle exits and no active collection or variable refers to its ticket, the ticket may become eligible for garbage collection.

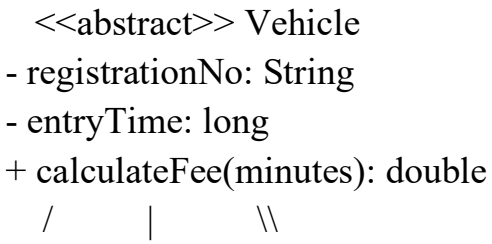
Static counter space is O(1) regardless of the number of tickets. Storing n active tickets requires O(n) space. Methods receive object references and can return objects or values; passing a reference does not automatically copy the object.

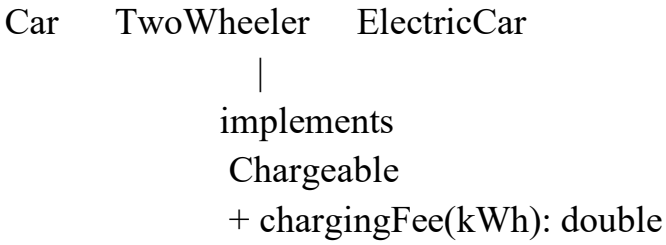
### TASK 6 — Design the Inheritance Hierarchy

Use an abstract Vehicle superclass for common fields and operations. Car, TwoWheeler and ElectricCar extend Vehicle. ElectricCar can implement Chargeable, which defines chargingFee(int kWh). This is hierarchical inheritance because several subclasses share the same parent. The superclass contains registration number, entry time and common validation; subclasses add category-specific fields or fee rules.

When an ElectricCar is constructed, Java first executes the Vehicle constructor through super(...), then executes the ElectricCar constructor body. This ensures the shared Vehicle state is initialised before the subclass state. Inheritance is suitable because each car is a Vehicle (an “is-a” relationship). If vehicle categories change frequently, composition with a VehicleType or FeePolicy object can be more flexible because a new rate can be added without creating another subclass.

### Simple UML hierarchy




Car
TwoWheeler
ElectricCar
implements
Chargeable
+ chargingFee(kWh): double